Terminal Snake: An Operating-Systems Case Study in Custom Memory, I/O, Process Management, and WebAssembly Cross-Platform Deployment

Navnit Naman
School of Technology
Rishihood University
Sonipat, India
230085

Kanhaiya Kumar
School of Technology
Rishihood University
Sonipat, India
230062

***Abstract*—This paper presents *Terminal Snake*, a real-time game written in ISO C99 that intentionally avoids the C standard library's memory, string, and math subsystems. The project serves as a self-contained demonstration of core operating-systems concepts: a best-fit heap allocator with bidirectional coalescing, raw-mode terminal I/O via POSIX `termios`, ANSI/VT100 screen rendering, a cooperative game-loop scheduler, and structured error-handling. All five OS-course evaluation domains are addressed. Beyond the terminal build, the project was ported to the browser via Emscripten and WebAssembly: the same game C source compiles to `.wasm` and runs inside an xterm.js terminal in any modern browser, with procedural sound generated by the Web Audio API and high-score persistence via `localStorage`. Two distinct game modes (Classic and Wrap) are implemented. A consecutive-food combo multiplier (up to $\times$8), a lives/respawn system, an obstacle system, and adaptive difficulty scaling make the gameplay rich while remaining fully implemented in hand-crafted C. Version history is maintained on GitHub under `kanhaiyak23/Snake_game_`.**

***Index Terms*—operating systems, memory management, process management, I/O management, custom allocator, terminal game, WebAssembly, Emscripten, Web Audio API, C99, POSIX, best-fit allocation, coalescing, cross-platform**

I. Introduction

Most hobby games lean on `malloc`, `printf`-wrappers, and framework event loops without considering the subsystems underneath. This project inverts that approach: the constraint of *no standard-library memory, string, or math functions* forces every OS primitive to be made explicit and auditable.

The five pillars addressed are:

- **Memory Management** — a 1 MiB static pool with best-fit allocation and bidirectional free-block coalescing.
- **I/O Management** — raw-mode keyboard capture via `termios` and ANSI/VT100 screen rendering; in the browser, replaced by JavaScript key-event injection and xterm.js output.
- **Process Management** — a deterministic cooperative game loop with `usleep`-based frame pacing (terminal) or `emscripten_set_main_loop` (browser) and level-dependent speed scaling.
- **Security / Error Handling** — structured termination codes, double-free guards, out-of-bounds checks, and graceful OOM recovery.
- **File Systems / Persistence** — high-score persistence via the browser's `localStorage` API; a plain-file equivalent is documented for the terminal build in future scope.

The codebase has two compilation targets that share all game logic but swap in platform-specific I/O implementations through the build system, demonstrating a *hardware abstraction layer* design without conditional compilation inside library headers.

II. System Architecture

A. Dual-Platform Module Layout

The project is divided into three layers: shared custom libraries, shared game logic, and platform-specific I/O drivers:

B. Platform Abstraction Pattern

All platform-specific functionality is isolated in leaf modules that implement the same function signatures declared in shared headers. The `Makefile` selects the correct set of `.c` files for each target:

- `make` links `keyboard.c + screen.c + sound.c + main.c` to produce a native POSIX binary (`./snake`).
- `make wasm` invokes `emcc` and links `keyboard_wasm.c + screen_wasm.c + sound_wasm.c + main_wasm.c` to produce `docs/snake.wasm` and `docs/snake.js`.

No `#ifdef` guards are needed in shared headers; the abstraction is achieved purely through the linker, exactly as a real OS hardware abstraction layer works.

TABLE I
SOURCE MODULE OVERVIEW

Module	Responsibility
Shared custom libraries (both targets)	
`math.c`	Arithmetic (mul, div, mod, abs, clamp), XorShift32 PRNG
`string.c`	strlen, strcpy, strcmp, itoa, strtok replacements
`memory.c`	1 MiB static pool, best-fit allocator, coalescing
`sound.h`	Sound API declaration (platform-independent)
Shared game logic (both targets)	
`snake.h/c`	Game structs, movement, collision, scoring, lives, obstacles, wrap mode
Terminal-specific (native build)	
`keyboard.c`	POSIX termios raw mode, escape-sequence state machine
`screen.c`	ANSI/VT100 renderer, TIOCGWINSZ layout
`sound.c`	Terminal beep stub (no audio hardware dependency)
`main.c`	Start screen, countdown, game loop, game-over screen
Browser-specific (Emscripten/WASM build)	
`keyboard_wasm.c`	JS key-event → C ring buffer bridge
`screen_wasm.c`	Output buffer flushed to xterm.js via `HEAPU8`
`sound_wasm.c`	Procedural synthesis via Web Audio API
`main_wasm.c`	State machine, localStorage high score, Emscripten loop
`test_logic.c`	Non-interactive unit tests for library correctness

C. Constraint Rationale

The project prohibits `<string.h>`, `<math.h>`, `malloc`, `free`, and `rand`. The only permitted standard-library symbols are `printf`/`putchar` (terminal I/O) and `exit` (process control). This constraint forces every memory allocation, random number, and string operation to be implemented and understood from first principles—precisely the pedagogical intent of an OS course project.

III. MEMORY MANAGEMENT

A. Pool Design

A single `static char VRAM[1 048 576]` array constitutes the entire heap. On `mem_init()`, it is formatted as one free `BlockHeader` spanning the whole array:

```
typedef struct BlockHeader {
    int  size;              /* user payload bytes */
    int  used;              /* 1=allocated, 0=free
        */
    struct BlockHeader *next;
} BlockHeader;
```

All blocks are linked in address order, enabling $O(1)$ adjacency tests needed for coalescing.

B. Best-Fit Allocation

On each `alloc(size)` call, the allocator:

1) Rounds `size` up to the nearest 8-byte boundary (alignment).
2) Scans the free list for the smallest block that fits (*best-fit* policy), breaking early on an exact match.
3) Splits the chosen block if the remainder exceeds sizeof(BlockHeader) $+ 8$ bytes.
4) Returns `NULL` on exhaustion, triggering the OOM error path.

Best-fit was chosen over first-fit because snake segments are all the same size (`sizeof(Segment)` = 12 bytes). Under uniform allocation sizes, best-fit leaves fewer unusable slivers [1].

C. Bidirectional Coalescing

On `dealloc(ptr)`, the allocator:

1) Validates the pointer is within `VRAM` (bounds guard).
2) Checks `blk->used` to prevent double-free corruption.
3) Marks the block free and coalesces *rightward* (absorbs consecutive free successors in a `while` loop).
4) Finds the predecessor by address-order traversal and coalesces *leftward* if the predecessor is free and adjacent.

This two-phase coalescing prevents fragmentation even under the alloc-one-grow/dealloc-one-shrink pattern of every game tick.

D. Per-Tick Allocation Pattern

The snake body is a singly-linked list of `Segment` nodes, each allocated from the pool. Every tick:

- `alloc(sizeof(Segment))` creates a new head at the computed next position.
- If no food was eaten, `dealloc(tail)` removes the oldest segment, keeping pool usage bounded at exactly `length × sizeof(Segment)` bytes.
- On food consumption the tail is *not* freed, growing the snake by one segment permanently.

E. Live Memory HUD

`mem_used()` walks the block list and sums used payload bytes. This value is displayed every frame in the HUD, making heap behaviour observable during gameplay.

IV. I/O MANAGEMENT

A. Terminal: Keyboard Input

`keyboard_init()` transitions the terminal into raw, non-canonical, non-echoing mode via POSIX `termios`:

```
new_tio.c_lflag &= ~(ICANON | ECHO);
new_tio.c_cc[VMIN]  = 0;  /* non-blocking */
new_tio.c_cc[VTIME] = 0;
tcsetattr(STDIN_FILENO, TCSANOW, &new_tio);
fcntl(STDIN_FILENO, F_SETFL, flags | O_NONBLOCK);
```

Setting `VMIN=0, VTIME=0` with `O_NONBLOCK` makes `read()` return immediately whether or not a character is waiting. This is necessary for a real-time game loop that cannot block on user input.

1) Escape-Sequence State Machine: Arrow keys arrive as three-byte ANSI sequences (`ESC [ A/B/C/D` for up-/down/right/left). `keyPressed()` implements a four-state machine ($esc_state \in \{0, 1, 2, 3\}$) to reassemble these sequences across successive `read()` calls and map them to the WASD key codes used by game logic. Stale buffered input is drained between screens to prevent bleed-through.

2) Terminal Restoration: `keyboard_restore()` removes `O_NONBLOCK` and calls `tcsetattr` with the saved `old_tio` snapshot, guaranteeing the terminal is left in a working state on both normal exit and error paths.

B. Browser: Keyboard Bridge

In the browser build, `keyboard_wasm.c` exports a C function `wasm_push_key(int code)` via `EMSCRIPTEN_KEEPALIVE`. The HTML page registers a `keydown` event listener that maps browser key codes (e.g. `ArrowUp`, `KeyW`) to ASCII characters and calls `wasm_push_key` via `Module.ccall`. The key is stored in a small ring buffer; `keyPressed()` dequeues one character per tick, replicating the drain semantics of the termios implementation exactly.

C. Terminal: Screen Output

`screen.c` is a thin wrapper around ANSI/VT100 escape sequences:

- **Alternate screen buffer** (`\033[?1049h/l`): isolates game rendering from the user's shell history.
- **Cursor control** (`\033[row;colH`): positions output anywhere on the terminal grid.
- **SGR color codes** (e.g., `\033[92m` for bright green): style the snake, food, obstacles, and HUD.
- **Unicode box-drawing** (): rendered as raw UTF-8 bytes via `screen_putstr`, replacing the earlier ASCII border.

`screen_update_layout()` queries the terminal dimensions via `ioctl(TIOCGWINSZ)` on every frame, allowing the game field to scale dynamically. All game code accesses field geometry through accessor functions (`screen_field_x/y/w/h()`) rather than cached constants.

D. Browser: Screen Output Buffer

`screen_wasm.c` maintains a 65 536-byte character buffer `obuf[]` in C memory. Every `screen_putchar` appends to this buffer. On `screen_flush()`, the entire buffer is transferred to JavaScript as a `Uint8Array` via Emscripten's `HEAPU8.slice()` view and passed to xterm.js's `term.write()` method. This batches all rendering into a single JS call per frame, minimising JS/WASM boundary crossings for performance.

V. WebAssembly Browser Build

A. Emscripten Compilation

The browser target is compiled with `emcc` (Emscripten C compiler), which translates C99 source to WebAssembly bytecode. The output is a `.wasm` binary loaded by a companion `.js` glue file. Both files are placed in `docs/` and served directly from GitHub Pages, making the game playable at any browser without installation.

Key Emscripten-specific features used:

- `EM_ASM{...}` — inline JavaScript executed synchronously from C, used for all Web Audio API calls and localStorage access.
- `EM_ASM_INT{...}` — inline JavaScript that returns an integer to C, used to read the stored high score from `localStorage`.
- `EMSCRIPTEN_KEEPALIVE` — prevents the C linker from dead-stripping functions that are called only from JavaScript (e.g. `wasm_push_key`).
- `emscripten_set_main_loop(tick, 10, 1)` — registers the game tick function with the browser's event loop at 10 Hz base rate, replacing the native `while(1) + usleep` pattern. The actual frame rate is adjusted per-tick via `emscripten_set_main_loop_timing`.

B. State Machine Architecture

`main_wasm.c` implements a four-state finite state machine that drives all game screens:

TABLE II
Browser Build State Machine

State	Screen	Transition
`STATE_START`	Splash / title screen	SPACE/ENTER → MENU
`STATE_MENU`	Mode selector	SPACE/ENTER → PLAYING
`STATE_PLAYING`	Game field	game_over → DEAD
`STATE_DEAD`	Game-over screen	SPACE → MENU, Q → START

The splash screen displays the ASCII art title, keyboard controls, and the combo system hint. The menu lets the player navigate between Classic Mode and Wrap Mode using W/S or arrow keys before confirming with SPACE/ENTER.

C. High-Score Persistence via `localStorage`

The browser build persists the session high score using the browser's `localStorage` key–value store:

```
static void hs_load(void) {
    high_score = EM_ASM_INT({
        return parseInt(
            localStorage.getItem('snakeHS') || '0',
                10);
    });
}
static void hs_save(void) {
    EM_ASM({
        try { localStorage.setItem('snakeHS', $0); }
        catch(e) {}
    }, high_score);
}
```

`hs_load()` is called once at startup; `hs_save()` is called only when a new high score is achieved, minimising storage writes. The `try/catch` in `hs_save` handles private-browsing environments where `localStorage` throws a `SecurityError`. The stored value is presented on

the game-over screen and highlighted with a "New Personal Best!" message when broken.

VI. Procedural Sound System

A. Design Rationale

The terminal build has no reliable, portable audio hardware path, so `sound.c` provides no-op stubs. The browser build implements a full procedural sound engine in `sound_wasm.c` using the Web Audio API (`AudioContext`), with no audio files or external assets.

The `AudioContext` is lazily created on first use and cached in `window._audioCtx`. If the context is suspended (browser autoplay policy), `ctx.resume()` is called before scheduling notes. Three internal note schedulers handle different wave types: `sched_sine`, `sched_square`, and `sched_saw`.

B. Implemented Sounds

TABLE III
Procedural Sound Events

Function	Description
`sound_eat(combo)`	Two-phase sine "coin blip". Base pitch starts at 660 Hz and rises 40 Hz per combo level (capped at 920 Hz). A perfect-fifth (3:2 ratio) upper note follows 35 ms later.
`sound_die()`	Three descending sawtooth tones: A4 (440 Hz) → E4 (330 Hz) → A3 (220 Hz), spaced 95 ms apart. Sawtooth timbre gives a harsh death-klaxon character.
`sound_levelup()`	Five-note ascending square-wave fanfare: G4→B4→D5→G5→D6, spaced 85 ms apart. Square wave produces the retro 8-bit aesthetic.
`sound_game_start()`	C major chord bloom: C4, G4, C5, G5 staggered by 60 ms each, using sine waves for a clean, warm attack.
`sound_wrap()`	FM synthesis portal wobble: an 8 Hz LFO (modulator) with 180 Hz depth drives a carrier sweeping from 420 Hz to 200 Hz over 200 ms, producing a warp-pipe warble.

C. FM Synthesis Implementation

`sound_wrap()` demonstrates FM (frequency modulation) synthesis, a technique normally requiring floating-point DSP:

```
var mod = ctx.createOscillator();    /* 8 Hz LFO */
var modGain = ctx.createGain();      /* depth: 180 Hz
     */
var car = ctx.createOscillator();    /* carrier:
     420->200 Hz */
mod.connect(modGain);
modGain.connect(car.frequency);      /* LFO drives
     carrier */
```

The modulator oscillator's output (scaled by the gain node) is routed directly into the carrier oscillator's `frequency` AudioParam, creating periodic pitch variation. The carrier simultaneously sweeps downward via `exponentialRampToValueAtTime`, combining the two effects into a single warbling tone.

VII. Process Management

A. Terminal: Game Loop Architecture

The terminal game runs under a cooperative, single-process loop in `main.c`:

1) `keyPressed()` — non-blocking poll for user input.
2) `game_handle_input()` — apply direction change or pause/quit.
3) `game_update()` — advance physics by one tick.
4) `game_render()` — redraw the entire frame.
5) `usleep(game_speed_delay())` — yield CPU for the remainder of the frame budget.

This is a textbook *fixed-timestep cooperative loop* [2]. There is no preemptive multitasking; the process voluntarily yields via `usleep`, which on macOS/Linux maps to `nanosleep` at the kernel level.

B. Browser: Emscripten Event Loop

In the browser build, `usleep` and `while(1)` are replaced by:

```
emscripten_set_main_loop(tick, 10, 1);
```

This registers `tick()` with the browser's `requestAnimationFrame` scheduler at a nominal 10 calls/second. The actual rate is adjusted per-tick:

```
emscripten_set_main_loop_timing(
    EM_TIMING_SETTIMEOUT,
    game_speed_delay(&gs) / 1000);  /*  s  -> ms */
```

This converts the same microsecond delay value used by the terminal build into milliseconds for the browser's `setTimeout`-based scheduler, giving both targets identical frame pacing without any game-logic changes.

C. Frame-Rate Scaling

`game_speed_delay()` computes the inter-frame sleep duration as a function of level:

$$\text{delay}(\ell) = \text{clamp}(180{,}000 - 15{,}000 \times (\ell - 1),\ 50{,}000,\ 180{,}000)\ \mu\text{s}$$

Level 1 yields a 180 ms frame period (5.6 ticks/s), and level 10 yields 50 ms (20 ticks/s). All arithmetic uses `my_mul` and `my_clamp` from the custom math library.

D. Level Progression

Level advances every 50 score points:

$$\ell = \min\left(\left\lfloor \frac{\text{score}}{50} \right\rfloor + 1,\ 10\right)$$

Because the score multiplier (Section IX-C) accelerates point gain, high-skill play naturally reaches maximum speed faster.

E. Startup Sequence

The ordered startup sequence enforces a safe initialisation contract: `mem_init` → `keyboard_init` → `screen_init` → `my_srand` → `hs_load` → game loop → `game_free` → `screen_cleanup` → `keyboard_restore`. Resource acquisition and release are strictly paired, preventing leaks across play-again sessions.

VIII. Security and Error Handling

A. Structured Termination Codes

Game-ending conditions are represented as symbolic constants rather than magic integers:

TABLE IV
Game-End Reason Codes

Constant	Trigger
`GAME_END_NONE`	Game still running
`GAME_END_COLLISION`	Wall, obstacle, or self-collision
`GAME_END_OOM`	`alloc()` returned `NULL`
`GAME_END_QUIT`	User pressed `Q`
`GAME_END_INTERNAL`	Linked-list invariant violated

B. Memory Safety

- **Bounds guard:** `dealloc()` checks the pointer lies within `VRAM`; out-of-range pointers are silently ignored.
- **Double-free guard:** the `used` flag is checked before marking a block free; a second `dealloc` on the same pointer is a no-op.
- **OOM propagation:** `alloc()` returning `NULL` is checked at every call site; the game transitions to `GAME_END_OOM` rather than dereferencing a null pointer.
- **Invariant assertion:** if the tail-search loop cannot locate the tail pointer, the tick is aborted and `GAME_END_INTERNAL` is set, preventing use-after-free.

C. Input Validation

- **Boundary check:** before placing the new head, `my_in_bounds()` verifies the position in Classic mode; in Wrap mode, modular arithmetic is applied instead of a bounds check.
- **Anti-reversal:** `game_handle_input()` blocks the 180-degree direction change by checking the current velocity before updating `dx`/`dy`.
- **Input draining:** the keyboard buffer is flushed between menu screens, preventing stale escape sequences from triggering unintended game actions.

D. Terminal Safety

Both `keyboard_restore()` and `screen_cleanup()` are always called on the exit path, ensuring the terminal is never left in raw mode or on the alternate screen buffer.

IX. Gameplay Features

A. Two Game Modes

The menu screen offers two distinct modes selectable before each game:

- **Classic Mode** — hitting any wall or obstacle costs a life and triggers respawn. Wall collision is detected with `my_in_bounds(nx, ny, field_w, field_h)`.
- **Wrap Mode** — the snake exits one edge and reappears on the opposite edge, using modular arithmetic:

```
/* Wrap mode: teleport through walls */
nx = my_mod(nx + field_w, field_w);
ny = my_mod(ny + field_h, field_h);
```

In Wrap mode, `sound_wrap()` is triggered on each wall crossing to give audio feedback for the teleportation. Self-collision and obstacle collision still end a life in Wrap mode.

B. Core Tick Loop

Each call to `game_update()` performs, in order: (1) layout refresh, (2) out-of-bounds food relocation, (3) head position computation, (4) boundary/wrap handling, (5) self-collision check, (6) obstacle collision check, (7) food collision detection, (8) new head allocation, (9) tail removal or retention, (10) score/level update. This strict ordering prevents read-after-write hazards within a single tick.

C. Score Multiplier (Combo System)

A consecutive-food combo system rewards sustained play without dying. The `combo` counter in `GameState` increments on each food eaten and resets to zero on any life loss. The score awarded per food is:

$$\Delta\text{score} = 10 \times \ell \times \min(\text{combo},\ M_{\max})$$

where ℓ is the current level and $M_{\max} = 8$ is the multiplier cap (`MAX_COMBO_MULT`). The HUD highlights the multiplier in bold yellow whenever it exceeds $1\times$, giving the player real-time streak feedback. The eat-sound pitch also rises with combo level, providing audio confirmation of the streak.

D. Lives System

The player begins with `INITIAL_LIVES` = 3 lives. `lose_life_and_respawn()` decrements the counter, resets the combo, clears all segment memory, rebuilds the snake at field centre, and repositions all food items. When lives reach zero, the game ends with `GAME_END_COLLISION`. Session state (score, high score, level) persists across respawns within one game.

E. Obstacle System

Each game session places a random number of obstacles on the field. Obstacles are statically allocated as an array `obstacles[MAX_OBSTACLES]` (with `MAX_OBSTACLES` = 10) inside `GameState`—no dynamic allocation is used. Placement uses the same rejection-sampling loop as food placement: a candidate position is rejected if it overlaps the snake, any food item, any existing obstacle, or the center

spawn area. In Classic mode, hitting an obstacle costs a life; in Wrap mode, the same rule applies since wall-wrapping does not protect against obstacles. Obstacles are drawn in a distinct colour to differentiate them from the snake body and food items.

F. Multi-Food Spawning

Up to `MAX_FOODS = 3` food items are active simultaneously. `place_food()` uses a rejection-sampling loop: a candidate position is drawn from `my_rand_range()` and rejected if it overlaps any snake segment, obstacle, or any other active food item. Since the snake is never allowed to fill the field (collision terminates the session first), the loop always terminates in finite expected time.

X. File System and Persistence

A. Browser: `localStorage` High Score

The browser build fully implements high-score persistence using `localStorage`, the browser's built-in key–value store that persists data across page refreshes and sessions for the same origin. The key used is `'snakeHS'`. The score is loaded once at startup and saved only when beaten, making the storage interaction lightweight.

B. Terminal: Deferred File I/O

The terminal build currently holds all game state in memory with no disk persistence. A minimal persistence layer would require:

- A fixed-size binary struct serialised to `~/.snake_score`.
- `open()`/`read()`/`write()`/`close()` calls (permitted as a POSIX hardware-abstraction exception).
- Atomic write-and-rename (`rename()` over a temporary file) to avoid partial writes on power loss.

This is documented as the highest-priority future item (Section XIII).

XI. Custom Math Library

A. Multiplication: Shift-and-Add

`my_mul(a, b)` uses the Russian-peasant (binary) method: the larger magnitude is doubled in each step while the smaller is halved; the larger is accumulated when the LSB of the smaller is 1. Time complexity is $O(\log \min(|a|, |b|))$ bit operations; space is $O(1)$.

B. Division: Recursive Doubling

`divide_positive_ll(a, b)` halves the problem by doubling the divisor until it exceeds the dividend, then unwinds the recursion accumulating the quotient in binary. Recursion depth is $O(\log(a/b))$. `my_div` wraps this with sign handling (truncation toward zero).

C. XorShift32 PRNG

The random number generator uses the XorShift32 algorithm [4]:

```
rng_state ^= rng_state << 13;
rng_state ^= rng_state >> 17;
rng_state ^= rng_state << 5;
```

This produces a full-period sequence of $2^{32} - 1$ values with no multiply in the hot path. It is seeded from `time(NULL)` at startup. Both the terminal and browser builds use the same PRNG; the browser seeds via `time()` which Emscripten maps to `Date.now()`.

XII. Results and Metrics

TABLE V
Implementation Metrics

Metric	Value
Total source lines (excl. blank/comments)	≈ 1,300
Shared game + library lines	≈ 870
Browser-specific (wasm) lines	≈ 430
Heap pool size	1 048 576 bytes (1 MiB)
Segment size (allocated unit)	12 bytes
Maximum live segments (level 10, length 200)	≈ 200
Peak heap usage at max length	≈ 4,800 bytes (0.46%)
Frame period range	50 ms – 180 ms
Max simultaneous food items	3
Max obstacles per session	10
Combo multiplier cap	8×
Procedural sound events	5
Game modes	2 (Classic, Wrap)
Unit tests passing	22 / 22

The allocator is empirically fragmentation-free for the snake workload: each tick allocates one segment and (on non-eat frames) frees one segment of identical size, so the pool converges to a near-steady-state after a few ticks.

XIII. Future Scope

1) **Terminal High-Score Persistence (File System):** Serialise the high score to a dot-file using POSIX `open/write/rename` with atomic replace semantics, matching the localStorage behaviour already implemented in the browser build.
2) **Slab Allocator:** Since all game allocations are the same size (`sizeof(Segment)`), a slab/pool allocator [3] would reduce per-allocation overhead from $O(n)$ (best-fit scan) to $O(1)$ (free-list pop).
3) **Configurable Difficulty:** Read a plain-text `.snakerc` configuration file at startup to set starting lives, field size, food count, and obstacle count.
4) **Replay Recording:** Log the RNG seed and input stream to a file; replay by re-seeding and feeding the recorded keystrokes, enabling deterministic replay without storing frame buffers.
5) **Multiplayer over Sockets:** Two game instances could share state via a UNIX-domain socket, exercising IPC and synchronisation primitives.

6) **Signal Handling:** Install a `SIGWINCH` handler to respond to terminal resize events asynchronously rather than polling `TIOCGWINSZ` every frame, reducing unnecessary system calls.
7) **IndexedDB Leaderboard:** Extend the browser build's high-score system to a full top-5 table using the browser's `IndexedDB` API, a structured alternative to `localStorage` for larger datasets.

XIV. Version History and Collaboration

All source code, commit history, and issue tracking are maintained at `github.com/kanhaiyak23/Snake_game_`. Contributions from both authors are individually attributable through commit authorship. Key milestones recorded in the history include:

- Initial snake movement and collision (`e1d954b`)
- Lives system with respawn (`fb0e676`)
- Multi-food spawning and HUD (`915acd4`)
- Input-parsing improvements and layout accessors (`205a8bc`)
- Score multiplier (combo system) and Unicode border (`766dafc`)
- WebAssembly build: Emscripten port with xterm.js renderer (`eaa3cdd`)
- Procedural sound engine via Web Audio API (`e6ce5e2`)
- Wrap mode, obstacle system, and browser state machine (`e6ce5e2`)

XV. Conclusion

Terminal Snake demonstrates that a complete, dual-platform game can be built atop hand-crafted OS primitives without sacrificing correctness or safety. The best-fit allocator with bidirectional coalescing correctly manages the alloc/dealloc pattern of the game loop under all tested conditions. The `termios` raw-mode keyboard driver and ANSI/VT100 renderer provide responsive I/O in the terminal with deterministic restoration. The Emscripten port proves that the platform-abstraction design—shared game logic, swappable I/O drivers—scales cleanly from a native binary to a WebAssembly module without modifying any game code. The procedural sound system, wrap-mode teleportation, obstacle placement, combo multiplier, and `localStorage` persistence collectively expand the project well beyond a minimal OS demonstration into a feature-complete, browser-playable arcade game. Every new feature is grounded in an OS concept: the sound engine exercises asynchronous scheduling; wrap mode applies modular arithmetic; persistence exercises the file-system module; and the state machine formalises process state transitions. The project provides a concrete, measurable reference implementation for each concept covered in the Operating Systems curriculum.

References

[1] D. E. Knuth, *The Art of Computer Programming, Vol. 1: Fundamental Algorithms*, 3rd ed. Reading, MA: Addison-Wesley, 1997, pp. 435–456.

[2] J. Gregory, *Game Engine Architecture*, 3rd ed. Boca Raton, FL: CRC Press, 2018, pp. 391–420.

[3] J. Bonwick, "The slab allocator: An object-caching kernel memory allocator," in *Proc. USENIX Summer Technical Conf.*, Boston, MA, Jun. 1994, pp. 87–98.

[4] G. Marsaglia, "Xorshift RNGs," *J. Statistical Software*, vol. 8, no. 14, pp. 1–6, 2003.

[5] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ: Wiley, 2018.

[6] M. Kerrisk, *The Linux Programming Interface*. San Francisco, CA: No Starch Press, 2010, pp. 1331–1360.

[7] IEEE and The Open Group, *The Open Group Base Specifications Issue 7, 2018 Edition (POSIX.1-2017)*, IEEE Std 1003.1-2017, 2017.

[8] A. Zakai, "Emscripten: An LLVM-to-JavaScript Compiler," in *Proc. ACM Int. Conf. Object Oriented Programming Systems Languages and Applications (OOPSLA)*, 2011, pp. 301–312.

[9] P. Adenot and H. Choi, *Web Audio API*, W3C Recommendation, Jun. 2021. [Online]. Available: https://www.w3.org/TR/webaudio/